

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Домашняя работа №4

Технический дизайн микросервисной архитектуры

Тема приложения: сайт поиска работы

Выполнила: Маркозубова Анастасия Крилловна

Проверил: Добряков Д. И.

Санкт-Петербург

2026 г.

1 Постановка задачи

Необходимо спроектировать разделение монолитного backend-приложения сайта поиска работы на микросервисы с использованием подхода database-per-service. Нужно определить границы сервисов, разделение базы данных, внутренние API для взаимодействия сервисов, возможные ошибки и шаги перехода от монолита к микросервисам.

2 Ход работы

Для постепенного перехода используется паттерн Strangler fig: новый функционал и отдельные домены постепенно выносятся из монолита в самостоятельные сервисы, а маршруты переключаются через API Gateway.

В текущем приложении можно выделить следующие независимые модули:

Сервис	Ответственность
Auth Service	регистрация, вход, JWT, пользователи
Profile Service	профили соискателей и компаний
Resume Service	резюме, образование, опыт работы
Vacancy Service	вакансии и поиск вакансий
Application Service	отклики на вакансии и статусы
Reference Service	отрасли и специализации

Каждый микросервис имеет собственную базу данных. Сервисы не обращаются напрямую к чужим таблицам. Если сервису нужны данные другого домена, он обращается к API соответствующего сервиса.

Разделение на микросервисы сайта поиска работы

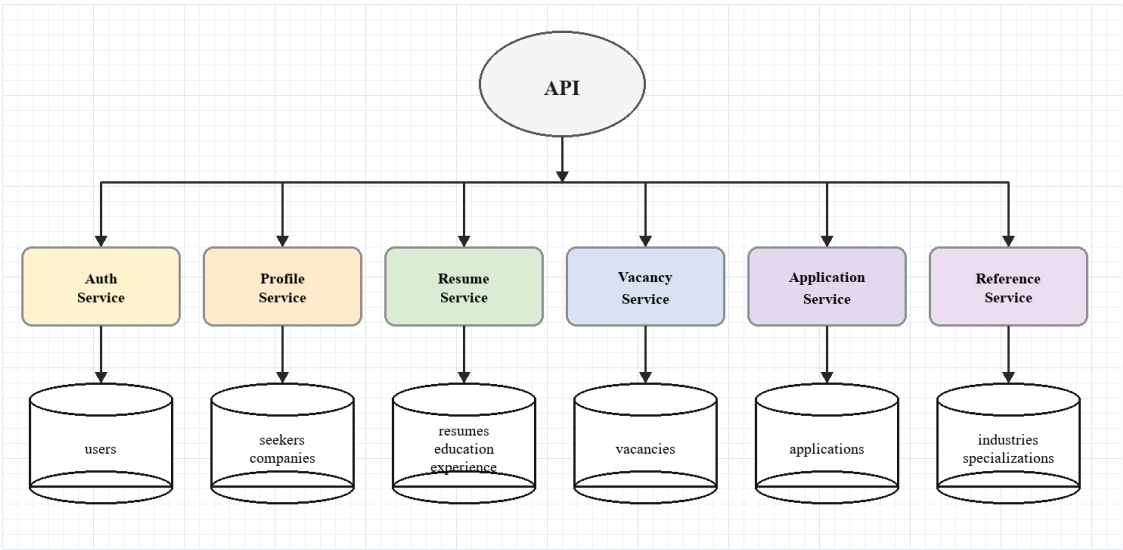


Рисунок 1 - БД, разделенная на микросервисы

Разделение базы данных:

База данных	Таблицы
auth_db	users
profile_db	seekers, companies
resume_db	resumes, educations, experiences
vacancy_db	vacancies
application_db	applications
reference_db	industries, specializations

Единые точки входа через API Gateway:

```
/api/auth/...  
/api/profiles/...  
/api/resumes/...  
/api/vacancies/...  
/api/applications/...  
/api/references/...
```

Эндпоинты для межсервисного взаимодействия:

Auth Service

```
GET /internal/users/{id}
```

Ответ: { "id": 1, "email": "user@mail.com", "role": "seeker" }

Ошибка: 404 user not found

Profile Service

```
GET /internal/seekers/by-user/{userId}
```

```
GET /internal/companies/by-user/{userId}
```

Ответ: { "seekerId": 3, "userId": 1, "firstName": "Иван", "lastName": "Иванов" }

Ошибка: 404 profile not found

Resume Service

```
GET /internal/resumes/{id}
```

```
GET /internal/resumes/{id}/ownership?seekerId=3
```

Ответ: { "resumeId": 5, "seekerId": 3, "title": "Backend developer" }

Ошибки: 404 resume not found, 403 not resume owner

Vacancy Service

```
GET /internal/vacancies/{id}
```

```
GET /internal/vacancies/{id}/ownership?companyId=2
```

Ответ: { "vacancyId": 7, "companyId": 2, "title": "Node.js developer" }

Ошибки: 404 vacancy not found, 403 not vacancy owner

Application Service

POST /internal/applications

Запрос: { "resumeId": 5, "vacancyId": 7, "seekerId": 3, "companyId": 2 }

Ответ: { "applicationId": 10, "status": "sent" }

Ошибки: 404 resume not found, 404 vacancy not found, 409 application already exists

Reference Service

GET /internal/specializations/{id}

Ответ: { "specializationId": 1, "name": "Backend" }

Ошибка: 404 specialization not found

Обработка ошибок:

Код	Описание
400	неверные данные запроса
401	пользователь не авторизован
403	нет прав на операцию
404	сущность не найдена
409	конфликт данных, например повторный отклик
500	внутренняя ошибка сервиса

Шаги перехода на микросервисы:

- выделить независимые домены в монолите
- описать маршрутизацию через API Gateway
- создать отдельные базы данных для сервисов
- вынести Reference Service и Auth Service
- вынести Profile, Resume, Vacancy и Application Service
- заменить прямые связи в БД на хранение внешних идентификаторов
- реализовать internal API между сервисами
- постепенно переключать маршруты с монолита на новые сервисы

3 Вывод

Монолитное приложение сайта поиска работы было разделено на микросервисы по бизнес-доменам. Для каждого сервиса определена собственная зона ответственности и отдельная база данных. Прямые связи между таблицами разных доменов заменены на хранение идентификаторов и взаимодействие через HTTP API.

Предложенная архитектура соответствует подходу database-per-service. Такой переход можно выполнять постепенно с помощью паттерна Strangler fig, не останавливая работу приложения. В результате систему становится проще развивать, тестировать и масштабировать по отдельным сервисам.